

CSC 472, Lab1 report by Alex Cooper

Step 1: Set up files in directory

Welcome, From: Cooper, Alex

```
● Connected Copy: Ctrl+Shift+C | Paste: Ctrl+Shift+V Disconnect

Linux lab-lab1-ac1029982 6.8.12-1-pve #1 SMP PREEMPT_DYNAMIC PMX 6.8.12-1 (2024-08-05T16:17Z)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Sep 13 15:15:58 2025
student@lab-lab1-ac1029982:~$ ls
course
student@lab-lab1-ac1029982:~$ cd course
student@lab-lab1-ac1029982:~/course$ ls
lab1 lab2 lab3 lab4
student@lab-lab1-ac1029982:~/course$ cd lab1
student@lab-lab1-ac1029982:~/course/lab1$ cp lab1.c ~/
student@lab-lab1-ac1029982:~/course/lab1$ cd ~
student@lab-lab1-ac1029982:~$ ls
course lab1.c
student@lab-lab1-ac1029982:~$ gcc lab1.c -o lab1 -m32 -no-pie
student@lab-lab1-ac1029982:~$ file lab1
lab1: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpret
er /lib/ld-linux.so.2, BuildID[sha1]=26d4444d0e7c9cd367d2bf0e1a8567d563e18f20, for GNU/Linux
3.2.0, not stripped
student@lab-lab1-ac1029982:~$
```

Step 2: gdb on lab1

```
student@lab-lab1-ac1029982:~$ gdb lab1
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef' to start, `gef config' to configure
93 commands loaded and 5 functions added for GDB 13.1 in 0.00ms using Python engine 3.11
Reading symbols from lab1...
(No debugging symbols found in lab1)
gef>
```

Step 3: disassemble the main()

```

gef> disas main
Dump of assembler code for function main:
0x0804916f <+0>:    lea     ecx,[esp+0x4]
0x08049173 <+4>:    and     esp,0xffffffff
0x08049176 <+7>:    push   DWORD PTR [ecx-0x4]
0x08049179 <+10>:   push   ebp
0x0804917a <+11>:   mov     ebp,esp
0x0804917c <+13>:   push   ebx
0x0804917d <+14>:   push   ecx
0x0804917e <+15>:   sub     esp,0x10
0x08049181 <+18>:   call   0x8049090 <__x86.get_pc_thunk.bx>
0x08049186 <+23>:   add     ebx,0x2e6e
0x0804918c <+29>:   mov     DWORD PTR [ebp-0xc],0x3
0x08049193 <+36>:   mov     DWORD PTR [ebp-0x10],0x4
0x0804919a <+43>:   push   DWORD PTR [ebp-0x10]
0x0804919d <+46>:   push   DWORD PTR [ebp-0xc]
0x080491a0 <+49>:   call   0x8049156 <multiply_by_two>
0x080491a5 <+54>:   add     esp,0x8
0x080491a8 <+57>:   mov     DWORD PTR [ebp-0x14],eax
0x080491ab <+60>:   sub     esp,0x8
0x080491ae <+63>:   push   DWORD PTR [ebp-0x14]
0x080491b1 <+66>:   lea     eax,[ebx-0x1fec]
0x080491b7 <+72>:   push   eax
0x080491b8 <+73>:   call   0x8049040 <printf@plt>
0x080491bd <+78>:   add     esp,0x10
0x080491c0 <+81>:   mov     eax,0x0
0x080491c5 <+86>:   lea     esp,[ebp-0x8]
0x080491c8 <+89>:   pop     ecx
0x080491c9 <+90>:   pop     ebx
0x080491ca <+91>:   pop     ebp
0x080491cb <+92>:   lea     esp,[ecx-0x4]
0x080491ce <+95>:   ret
End of assembler dump.
gef>

```

Q1: Identify the assembly instructions for creating the stack frame of the main() function.

The actual code that creates the stack frame begins at <+4>,

0x08049173 <+4>: and esp, 0xffffffff

0x08049176 <+7>: push DWORD PTR [ecx-0x4]

0x08049179 <+10>: push ebp

0x0804917a <+11>: mov ebp, esp

0x0804917c <+13>: push ebx

0x0804917d <+14>: push ecx

0x0804917e <+15>: sub esp, 0x10

This entire snippet is the entire creation of the stack frame.

Q2: Identify and explain the purpose of the two lines related to setting variables p and q.

The lines that set the variables are

0x0804918c <+29>: mov DWORD PTR [ebp-0xc],0x3

0x08049193 <+36>: mov DWORD PTR [ebp-0x10],0x4

The purpose of the code is to move the data to the specific memory location it should be in. DWORD PTR establishes that the variable incoming will have 4 bytes worth of memory to occupy. [ebp-0xc] and [ebp-0x10] are the actual addresses that are going to be used for the variables. 0x3 and 0x4 are the values 3 and 4 respectively and will be going into the address.

Q3: Before calling multiply_by_two(), why does the stack contain two sets of “3,4” instead of just one set (see Figure 1)?

The stack contains 2 sets of “3,4” because it now has a copy due to the function call. As previously mentioned, the variables were set which created one set. But since the variables need to be used in a function, in this case multiply_by_two() copies are produced to be placed into the function generating another set. This way of handling the variables keep the originals isolated in the event they may need to be used for something else and maintains consistency across the arguments.

Step 4: Disassemble the multiply_by_two()



```
gef> disas multiply_by_two
Dump of assembler code for function multiply_by_two
:
0x08049156 <+0>:    push    ebp
0x08049157 <+1>:    mov     ebp,esp
0x08049159 <+3>:    call    0x80491cf <__x86.get_pc_thunk.ax>
0x0804915e <+8>:    add     eax,0x2e96
0x08049163 <+13>:   mov     edx,DWORD PTR [ebp+0x8]
0x08049166 <+16>:   mov     eax,DWORD PTR [ebp+0xc]
0x08049169 <+19>:   add     eax,edx
0x0804916b <+21>:   add     eax,eax
0x0804916d <+23>:   pop     ebp
0x0804916e <+24>:   ret
End of assembler dump.
gef>
```

Q4: Explain the meaning of add eax, edx and add eax, eax. Why not use the mul (Multiply) instruction instead (take a guess)?

Likely because the add function is more efficient for this simple calculation. In our case we used add two times. The calculation being done is always going to require at least two steps to complete, combining the ints being one step, then the multiplication being the second. So, replacing either or both adds with mul will create a slower process.

Q5: Which register is used to store the final multiplication result?

The register used to store the final result is eax because the lines

0x08049169 <+19>: add eax, edx

0x0804916b <+21>: add eax, eax

Which show two values being added than a value being added to itself, the same as multiplying by two. The address used to store the values is the leading memory space eax.